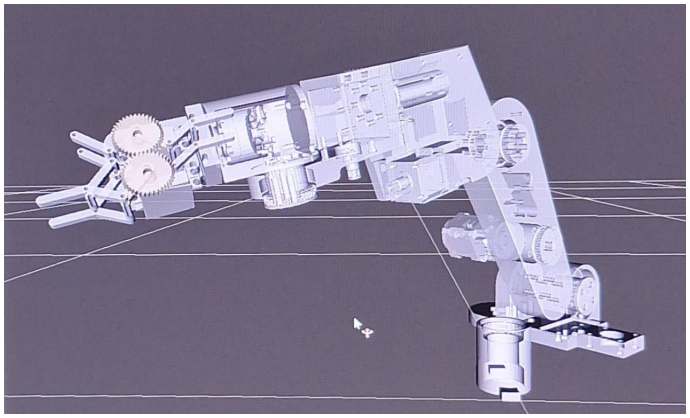
Entwicklung eines 6dof Roboterarms



ASIT-Projekt verfasst von

Matteo Gassler
Emanuel Käfer
Alexander Dick

Betreuer: Prof. Dr. Gerhard Gaube

HTBLA Saalfelden

Abteilung für Elektrotechnik

Abgabevermerk:

Saalfelden, 2 April 2026 übernommen von:

Vorwort

Vorwort Einfügen

--

Ort Datum Einfügen

Matteo Gassler & Emanuel Käfer & Alexander Dick

Aufgabenstellung

Da die Entwicklung unseres 6dof Roboterarms im Rahmen des Faches ASIT erfolgt und der Lernerfolg im Vordergrund steht konnten wir uns die Aufgabenstellung selbst stellen und hatten dabei keine großen Einschränkungen. Wir haben uns dazu entschieden das der Roboterarm als Mindestanforderung eine Pick and Place Operation, also ein Objekt aufheben und wo anders platzieren können soll. Als optionale weitere Herausforderung soll man über ein Graphical user Interface Koordinaten eingeben können zu denen sich der Arm dann bewegt.

--

Ort Datum Einfügen
Matteo Gassler & Emanuel Käfer & Alexander Dick

Inhaltsverzeichnis

Vorwort ii

Aufgabenstellung iii

1 Einleitung 1

2 Software 2

2.1 Softwarearchitektur und Steuerung 2

3 Hardware 7

4 Elektrisches 8

Quellenverzeichnis 10

1 Einleitung

2 Software

2.1 Softwarearchitektur und Steuerung

Dieses Kapitel beschreibt die Implementierung der zentralen Robotersteuerung, welche das funktionale Bindeglied zwischen der mechanischen Konstruktion (siehe **??**) und der elektrischen Leistungselektronik (siehe **??**) bildet. Die Firmware wurde in Bare-Metal-C für einen STM32F446-Mikrocontroller entwickelt. Ziel der Softwarearchitektur ist es, harte Echtzeitanforderungen für die synchrone Ansteuerung von sechs Schrittmotoren zu erfüllen und gleichzeitig eine hochgradig skalierbare Basis für zukünftige Pfadplanungen zu schaffen. Im Folgenden werden die Hardware-Abstraktion, die SPI-Treiberkonfiguration, das Konzept des Zustandsautomaten sowie die algorithmische Umsetzung der kinematischen Referenzierung detailliert erläutert.

2.1.1 Implementierung der 6 achsen und Skalierbarkeit

Bei der Ansteuerung eines 6-Achs-Roboterarms führt die Zuweisung von Mikrocontroller-Pins als Einzelnde Variablen und die Statische Zuweisung von Peripherie-Registern schnell zu Eingeschränkten und fehleranfälligem Programmcode . Um dies zu vermeiden, wurde eine Array-basierten Datenstruktur implementiert. Anstatt jede Achse separat zu adressieren, werden Pointer auf die entsprechenden Hardware-Timer (TIM HandleTypeDef), PWM-Kanäle sowie die GPIO-Ports für Richtungs- (DIR) und Chip-Select-Signale (CS) in globalen Arrays gekapselt. Durch diese Entkopplung von physikalischer Hardware und Steuerungslogik kann jede Achse über einen simplen Index angesprochen werden. Dies ermöglicht es, Kernfunktionen wie das Starten und Stoppen der Motoren oder das Homen über alle Treiber mittels einfacher Schleifenkonstrukte umzusetzen, was die Skalierbarkeit und Wartbarkeit des Systems massiv erhöht.

2.1.2 Initialisierung und SPI-Kommunikation der Motortreiber

Die verwendeten Schrittmotortreiber vom Typ TMC5160 in Kombination mir NEMA-17 Schrittmotoren erfordern eine initiale Konfiguration ihrer internen Register, um Parameter wie den Haltestrom, Run-Current und den Chopper-Modus festzulegen. Diese Kommunikation erfolgt über einen geteilten Serial Peripheral Interface (SPI) Bus. Um die Treiber individuell adressieren zu können, wird vor jedem Datentransfer der spezifische Chip-Select (CS) Pin der Zielachse auf ein niedriges Potential (LOW) gezogen. Um fehler direkt beim Initialisieren der Treiber zu erkennen und nicht erst später wurde eine Validierung eingebaut ohne die der nächste Schritt nicht erreicht werden kann. Nach dem Schreiben der Konfigurationsdaten liest der Mikrocontroller das globale Statusregister (GSTAT) sowie aktuelle Treiberzustände zurück. Nur wenn das System valide Antworten (GSTAT = 0) detektiert, wird die Steuerung freigegeben. Störsignale oder fehlende Hardware-Verbindungen werden so unmittelbar beim Systemstart erkannt und führen zu einem sicheren Systemabbruch.

Die gewählte Konfiguration der Motortreiber ist wie folgt:

Tabelle 2.1: Konfiguration der TMC5160T Register via SPI

Register	**Wert**	**Funktion**
`GSTAT` (0x01)	`0x00000007`	Setzt das globale Statusregister zurück. Löscht Fehlerspeicher für Reset, Unterspannung der Charge-Pump und Driver-Shutdown vor dem Testlauf.
`GCONF` (0x00)	`0x00000004`	Aktiviert das Bit `pdn_disable`. Schaltet den Treiber in den internen Sensing-Modus um und erzwingt die Kommunikation rein über die SPI-Schnittstelle.
`IHOLD_IRUN` (0x10)	`0x00061008`	Konfiguriert den Motorstrom: Setzt den Fahrstrom (`IRUN`) auf ca. 50 %, den Haltestrom im Stillstand (`IHOLD`) auf ca. 25 % sowie eine Verzögerung (`IHOLDDELAY`) für die Stromabsenkung.
`CHOPCONF` (0x6C)	`0x14010053`	Konfiguration des Chopper-Verfahrens. Aktiviert den kraftvollen *SpreadCycle*-Modus, stellt die Schrittauflösung auf 16 Mikroschritte (mit automatischer Interpolation auf 256 Schritte) und definiert das Signal-Timing (*TOFF*, *TBL*).
`PWMCONF` (0x70)	`0xC10D0024`	Hinterlegt die Parameter für den flüsterleisen *StealthChop2*-Modus (Spannungsamplitude, Frequenz, Gradienten) für einen potenziellen, geschwindigkeitsabhängigen Moduswechsel.

2.1.3 Finite State Machine(Endlicher Zustandsautomat)

Um die auf dem Mikrocontroller laufende Software übersichtlich zu gestalten und ein sich selbst blockieren der Software zu verhindern kommt eine Finite State Machine zum Einsatz. Eine Finite State Machine ist ein mathematisches Modell für Systeme, die sich zu jedem Zeitpunkt in genau einem von mehreren Zuständen befinden. Eine Finite State Machine besteht immer aus den Folgenden Komponenten:

- Zustände (States): Die verschiedenen Phasen des Systems
- Startzustand (initial State): Der Zustand in dem die FSM beginnt
- Übergänge (Transitions): Die Bedingungen für den Wechsel von Zuständen
- Ereignis (Inputs): Die Ereignisse die einen Wechsel bewirken
- Aktionen (Outputs): Die Ausgaben der FSM

Zustände

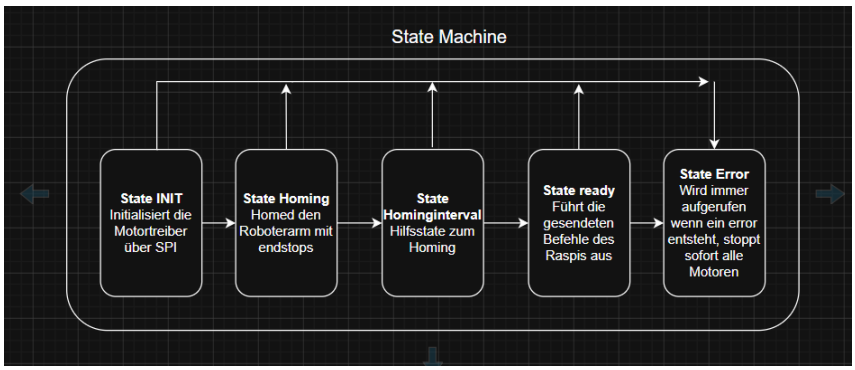


Abbildung 2.1: Diagramm der Finite State Maschine

Für unseren Roboterarm haben wir die Zustände STATE INIT, STATE HOMING, STATE HOMING OFFSET, STATE READY und STATE ERROR gewählt. Als Startzustand ist STATE INIT definiert da am Anfang bevor irgendetwas anderes passiert die Treiber initialisiert werden müssen.
Die Funktionen der Zustände gestaltet sich wie folgt:

2.1.4 Kinimatische Referenzierungsfahrt (Homing)

Da Schrittmotoren über keine absolute Positionscodierung verfügen und wir uns aus Preisgründen gegen ein Positionscodierung entschieden haben, ist der Systemzustand nach jedem Neustart undefiniert. Um den vordefinierten kinematischen Nullpunkt des

Tabelle 2.2: Funktionen der Zustände der Finite State Machine

Zustände	Funktion
STATE INIT	Initialisiert die Motortreiber über SPI, um die nötigen Parameter zu konfigurieren.
STATE HOMING	Hier passiert der erste Teil der kinematischen Referenzfahrt, die den Roboter in die Home-Position bringt. Er fährt dabei den Endschalter der jeweiligen Achse an.
STATE HOMING OFFSET	Hier passiert der zweite Teil der kinematischen Referenzfahrt. Der Roboter fährt vom Endschalter in die vordefinierte Home-Position.
STATE READY	Hier passiert der eigentliche Betrieb des Roboters. Der Mikrocontroller empfängt Befehle und führt diese aus, um den Roboter wie gewünscht zu bewegen.
STATE ERROR	In diesen Zustand gelangt man nur im Fehlerfall. Es werden alle Motoren gestoppt und eine Fehlermeldung ausgegeben.

Roboters zu ermitteln, muss zwingend eine Kalibrierungsfahrt (Homing) durchgeführt werden.

Interrupt-Verarbeitung und Hardware-Entprellung

Für die Erfassung der Endschalter kommen Externe Interrupts (EXTIS) zum einsatz. EXTIS sparen sehr viel Rechenaufwand und Energie da der Prozessor nicht permanent einen Pin überwachen muss, sondern sobald eine Flanke erkannt wird(Rising edge von 0 auf 3.3V oder Falling edge von 3.3V auf 0) wird ein Interrupt request ausgelöst der das Hauptprogramm pausiert und eine Spezielle Funktion aufruft. Diese Funktion setzt eine Flag um dem Hauptprogramm das der Endschalter getroffen wurde, das zu fahrende Offset zu berechnen ist und die Fahrtrichtung umgekehrt werden muss.
Um das mehrfache auslösen des interrupt zu verhinder das durch sogenantes Bouncen bei Mechanischen Schaltern auftreten kann ist in der Interrupt Funktion ein debouncing eingebettet das nach einem Interrupt alle weiteren für 30ns ignoriert. Der Code dafür sieht wie folgt aus:

```
1     static uint32_t last = 0;
2 uint32_t now = HAL_GetTick();
3 if (now - last < 30) return;
4 last = now;
```

Programmcode 2.1: Debouncing für Interrupt

Berechnung kinematischer Offsets

Die physische Position der Endschalter stimmt konstruktionsbedingt selten mit der gewünschten kinematischen Nullstellung () des Robotermodells überein. Daher wurde eine dynamische Offset-Berechnung implementiert. Sobald ein Endschalter auslöst, berechnet die Software die Differenz zwischen dem aus dem CAD-Modell bekannten Schalterwinkel () und dem kinematischen Nullpunkt () im Bogenmaß (Radiant). Dieser Winkeldifferenzwert wird anschließend in diskrete Motorschritte umgerechnet. Die Formel berücksichtigt dabei die Motorauflösung (200 Schritte/Umdrehung), das eingestellte Microstepping (1/16) sowie die mechanische Getriebeübersetzung von 1:20.

Sequenzieller Homing-Ablauf

Die Referenzierung des 6-Achs-Systems erfolgt sequenziell, gesteuert durch eine globale Index-Variable (current homingaxis), beginnend bei der Basis (Achse 0) bis zum Endeffektor (Achse 5). Der Ablauf für eine Achse gliedert sich dabei in folgende Phasen: Zunächst verfährt die Achse im Zustand STATE HOMING mit konstanter Geschwindigkeit auf den Endschalter zu. Nach Auslösen des Interrupts berechnet das System den erforderlichen Offset, invertiert das Richtungssignal (DIR) und wechselt in den Zustand STATE HOMING OFFSET. Erst wenn die exakt berechnete Schrittzahl für den Offset über den Timer-Interrupt abgefahren wurde, gilt die Achse als referenziert. Der Index wird inkrementiert und die Prozedur für das nächste Gelenk gestartet, bis das System vollständig kalibriert ist und in den Betriebszustand STATE READY übergeht.

3 Hardware

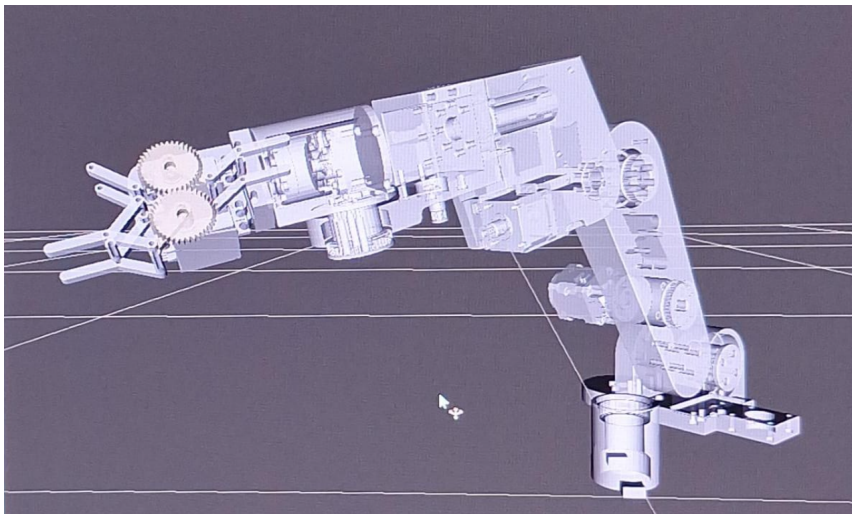


Abbildung 3.1: Blockdiagramm des 6-DOF-Roboterarms

4 Elektrisches

Die Entwicklung moderner Roboterarme basiert stark auf den Prinzipien der kollaborativen Robotik,[1] bei denen Sicherheit und Flexibilität im Vordergrund stehen [2].

Abbildungsverzeichnis

2.1 Diagramm der Finite State Maschine 4

3.1 Blockdiagramm des 6-DOF-Roboterarms 7

Quellenverzeichnis

[1] *WS2812B Intelligent control LED integrated light source Features and Benefits*. Adresse: https://cdn-shop.adafruit.com/datasheets/WS2812B.pdf

[2] *WS2812 Ansteuerung – Mikrocontroller.net*, Mikrocontroller.net, 2025. besucht am 10. Feb. 2025. Adresse: https://www.mikrocontroller.net/articles/WS2812_Ansteuerung?